Verificación facial con una red siamesa y pocas referencias

Karen Sofía Delgado Calderon

Universidad de Cundinamarca

Especialización en Inteligencia Artificial

Deep Learning – Conceptos

R2-A1-S10 Redes Neuronales V

5 de noviembre de 2026

Verificación facial con una red siamesa y pocas referencias

Una fotografía de referencia puede ser insuficiente cuando cambian la pose, la iluminación y la expresión. Esta práctica mide cuánto aporta una red siamesa frente a una comparación directa de píxeles y cuánto cambia el resultado al pasar de una a cinco referencias por identidad. La prueba usa fotografías públicas de *Labeled Faces in the Wild* (LFW) e identidades que quedaron fuera del ajuste. Con cinco referencias, la red acierta el 43,29 % de las consultas, frente al 33,54 % de la línea base.

Corpus y preprocesamiento

LFW reúne fotografías faciales públicas en condiciones variadas (Huang et al., 2007). Se usó la variante alineada *funneled* distribuida por el cargador de scikit-learn, con recorte facial y conversión a grises (Scikit-learn developers, s. f.). Al pedir al menos 20 imágenes por identidad se obtuvieron 3.023 recortes de 62×47 píxeles. Para reducir la diferencia de peso entre identidades, se seleccionaron entre 20 y 24 recortes por clase. Antes de seleccionar se verificaron copias exactas mediante SHA-256 sobre el recorte cargado.

Cada recorte se redujo a 40×30 por área, conservando aproximadamente su proporción. Se restó su intensidad media y se dividió por su desviación estándar. Esta operación reduce diferencias globales de brillo y contraste y conserva la estructura espacial de la cara. Puede perderse detalle al reducir la resolución. Las imágenes ya llegan alineadas y recortadas, de modo que esta práctica empieza después de la detección y el alineamiento originales.

Tabla 1
Inventario por fase del experimento

Fase	Identidades	Fotos	Uso
Ajuste	40	929	Aprender pesos
Calibración	10	234	Elegir umbrales
Prueba	12	280	Medir desempeño
Total	62	1.443	

Nota. La partición por identidad se generó con semilla 487. Una identidad se asigna a una sola fase. Los recuentos provienen de `resumen_lfw.json`.

La Tabla 1 separa por completo las identidades de cada fase. Así, la prueba mide transferencia a personas ausentes durante el ajuste y la calibración. Los archivos de entrega contienen el código de descarga, resultados agregados y la fuente del corpus. La procedencia pública se documenta con la referencia al conjunto de investigación.

Codificador y función de costo

Las ramas de la red siamesa comparten un mismo codificador. Tres capas convolucionales con filtros de 3×3 producen 16, 32 y 48 canales, con ReLU y dos reducciones espaciales mediante máximo. El promedio adaptativo entrega un mapa de $48 \times 4 \times 3$, que una capa lineal proyecta a 32 dimensiones. La normalización de norma dos deja cada vector en una esfera unitaria. El modelo tiene 37.136 parámetros entrenables.

El entrenamiento reúne un ancla a, otra toma p de la misma identidad y una toma n de una identidad distinta. La pérdida penaliza tríos donde la distancia positiva queda demasiado cerca de la negativa, siguiendo el principio de FaceNet (Schroff et al., 2015). Con d como distancia euclidiana entre representaciones y margen 0,30, la pérdida por trío es

$$L(a, p, n) = \max\{0,\ d(a, p) - d(a, n) + 0{,}30\}.$$

Se usó Adam con tasa 0,001 durante 24 épocas, con ocho lotes de 48 tríos por época. Cada positivo es una captura distinta del ancla. Las tres entradas pasan por los mismos pesos y la pérdida media del lote dirige el ajuste. El código completo se incluye en el apéndice.

Protocolo de reconocimiento y verificación

Cada episodio reserva cinco tomas por identidad para la galería y diez tomas distintas como consultas. En *one-shot* se emplea la primera toma de la galería. En *few-shot* se forma un prototipo con el promedio normalizado de las cinco representaciones, siguiendo la idea de prototipos de Snell et al. (2017). La línea base aplica la misma regla de galería a los 1.200 píxeles normalizados. Ambas representaciones reciben exactamente las mismas consultas.

El reconocimiento cerrado elige el prototipo más cercano entre las identidades de prueba. La verificación aplica un umbral de distancia a cada pareja consulta–prototipo. Se calibró un umbral por método y número de referencias con diez episodios, maximizando la diferencia entre la tasa de verdaderas aceptaciones y la tasa de falsas aceptaciones. Luego se congeló el umbral y se evaluaron veinte episodios sobre las 12 identidades reservadas.

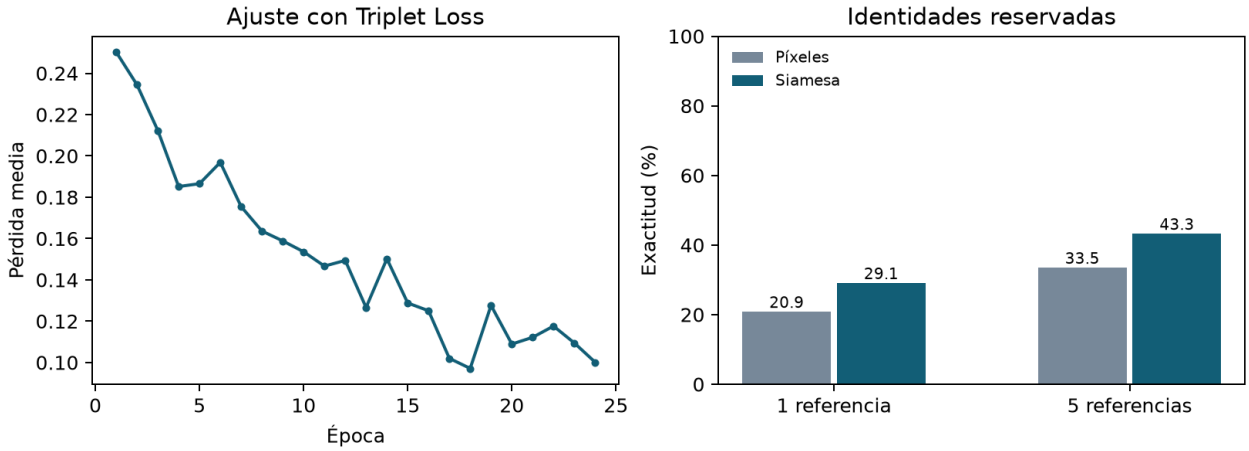
En verificación, FAR divide las falsas aceptaciones por las comparaciones entre identidades distintas. FRR divide los falsos rechazos por las comparaciones de la misma identidad. El área bajo la curva ROC (AUC) resume el orden de distancias al variar el umbral. *Zero-shot* requiere asignar una clase sin fotos de referencia usando información auxiliar. Aquí las identidades de prueba son nuevas para el codificador, pero cuentan con una galería y se evalúan como *one-shot* y *few-shot*.

Resultados

La pérdida media pasó de 0,2503 a 0,1001. La Figura 1 muestra las fluctuaciones durante el ajuste y la diferencia entre los dos métodos en prueba. Con una referencia la red supera a los píxeles por 8,25 puntos porcentuales. Con cinco, la diferencia es de 9,75 puntos. La exactitud de la red sube 14,17 puntos al ampliar la galería.

Figura 1

Pérdida de entrenamiento y exactitud en identidades reservadas



Nota. La curva muestra la media de ocho lotes por época. Las barras resumen 20 episodios por condición. Elaboración propia a partir de `resumen_lfw.json`.

Tabla 2

Desempeño en reconocimiento cerrado y verificación

Método	*k*	Aciertos	Exactitud	FAR	FRR	AUC
Píxeles	1	501	20,88 %	45,71 %	40,75 %	0,604
Píxeles	5	805	33,54 %	46,57 %	29,50 %	0,679
Siamesa	1	699	29,12 %	31,43 %	31,37 %	0,753
Siamesa	5	1.039	43,29 %	24,32 %	23,92 %	0,841

Nota. k indica referencias por identidad. Cada fila reúne 2.400 consultas y 26.400 comparaciones entre identidades distintas. Las fotos pueden reutilizarse entre episodios. Fuente: `resumen_lfw.json`.

La Tabla 2 muestra que el modelo separa mejor las parejas en ambas configuraciones. Con cinco referencias y umbral 0,7126, hubo 6.421 falsas aceptaciones de 26.400 comparaciones negativas y 574 falsos rechazos de 2.400 positivas. Con una referencia el umbral fue 0,8469. La reducción de ambos errores en este experimento favorece el uso de varias tomas, aunque la comparación conserva un margen de error considerable.

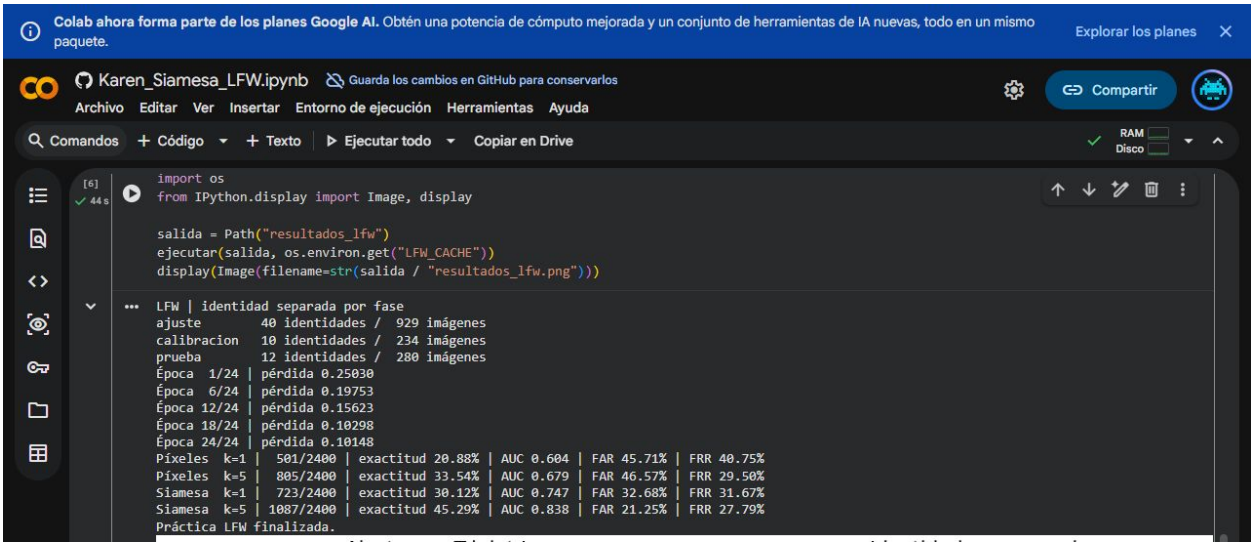
Reproducción y conclusiones

El repositorio GitHub de Karen reúne el código, las dependencias y los resultados. El cuaderno en Google Colab descarga LFW mediante scikit-learn. Para repetir la práctica en CPU, elija *Entorno de ejecución* y *Ejecutar todo*.

La Tabla 2 corresponde a la corrida local. En Colab se obtuvieron 723 y 1.087 aciertos, equivalentes a 30,12 % y 45,29 % con una y cinco referencias. La línea base obtuvo los mismos recuentos en ambos entornos. La Figura 2 muestra la finalización del cuaderno y las métricas de esa segunda corrida.

Figura 2

Ejecución del experimento LFW en Google Colab



Nota. Captura propia del 5 de noviembre de 2026. El entorno de Colab completó las seis celdas de código. Los valores de la red varían entre la corrida de Colab y la local.

La red mejoró la exactitud cerrada y el AUC frente a la línea base, con el mejor resultado al usar cinco referencias. La muestra favorece identidades con muchas fotos. Los episodios reutilizan imágenes y se evaluó una sola partición, por lo que las cifras describen este protocolo en LFW.

Referencias

Huang, G. B., Ramesh, M., Berg, T., & Learned-Miller, E. (2007). *Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments* (Technical Report No. 07-49). University of Massachusetts Amherst. https://people.cs.umass.edu/~elm/papers/lfw.pdf

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. En *2015 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 815–823). IEEE. https://doi.org/10.1109/CVPR.2015.7298682

Scikit-learn developers. (s. f.). *fetch_lfw_people*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_lfw_people.html

Snell, J., Swersky, K., & Zemel, R. S. (2017). Prototypical networks for few-shot learning. *Advances in Neural Information Processing Systems, 30*. https://papers.nips.cc/paper_files/paper/2017/hash/cb8da6767461f2812ae4290eac7cbc42-Abstract.html

Apéndice

Código de la práctica facial

```
"""Practica de comparacion facial en el conjunto cerrado LFW."""
from pathlib import Path
import hashlib
import json

import matplotlib.pyplot as plt
import numpy as np
import torch
from sklearn.datasets import fetch_lfw_people
from sklearn.metrics import roc_auc_score, roc_curve
from torch import nn
from torch.nn import functional as F

SEMILLA = 487
EPOCAS, LOTES, TRIOS = 24, 8, 48
MARGEN = 0.30
torch.set_num_threads(2)
torch.manual_seed(SEMILLA)
rng = np.random.default_rng(SEMILLA)

# %% Datos y particion
def preparar(datos):
    """Conserva luminancia y comprueba copias exactas del recorte."""
    grupos, vistas = {}, set()
    for clase in np.unique(datos.target):
        indices = rng.permutation(np.flatnonzero(datos.target == clase))
        seleccion = []
        for idx in indices:
            huella = hashlib.sha256(datos.images[idx].tobytes()).digest()
            if huella not in vistas:
                vistas.add(huella)
                seleccion.append(idx)
            if len(seleccion) == 24:
                break
        if len(seleccion) >= 20:
            grupos[int(clase)] = np.array(seleccion)
    imagenes = torch.as_tensor(datos.images, dtype=torch.float32).unsqueeze(1)
    imagenes = F.interpolate(imagenes, (40, 30), mode="area")
    media = imagenes.mean(dim=(2, 3), keepdim=True)
    desvio = imagenes.std(dim=(2, 3), keepdim=True).clamp_min(1e-6)
    return (imagenes - media) / desvio, grupos


def partir(grupos):
    clases = rng.permutation(list(grupos))
    ajuste, calibracion, prueba = clases[22:], clases[:10], clases[10:22]
    if len(ajuste) < 2:
```

```
        raise ValueError("LFW no contiene suficientes identidades elegibles.")
    return dict(ajuste=ajuste, calibracion=calibracion, prueba=prueba)


# %% Codificador y ajuste
class Codificador(nn.Module):
    def __init__(self):
        super().__init__()
        self.capas = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 48, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((4, 3)), nn.Flatten(), nn.Linear(576, 32),
        )

    def forward(self, imagenes):
        return F.normalize(self.capas(imagenes), dim=1)


def ajustar(modelo, imagenes, grupos, clases):
    optimizador = torch.optim.Adam(modelo.parameters(), lr=0.001)
    criterio = nn.TripletMarginLoss(margin=MARGEN)
    historial = []
    for epoca in range(EPOCAS):
        costos = []
        for _ in range(LOTES):
            anclas, positivos, negativos = [], [], []
            for _ in range(TRIOS):
                clase, otra = rng.choice(clases, 2, replace=False)
                a, p = rng.choice(grupos[clase], 2, replace=False)
                anclas.append(a)
                positivos.append(p)
                negativos.append(rng.choice(grupos[otra]))
            optimizador.zero_grad()
            z = modelo(imagenes[anclas + positivos + negativos])
            costo = criterio(*z.chunk(3))
            costo.backward()
            optimizador.step()
            costos.append(costo.item())
        historial.append(float(np.mean(costos)))
        if epoca == 0 or (epoca + 1) % 6 == 0:
            print(f"Epoca {epoca + 1:2d}/{EPOCAS} | perdida {historial[-1]:.5f}", flush=
    True)
    return historial


# %% Evaluacion fuera del ajuste
def episodios(clases, grupos, semilla, cantidad):
    generador = np.random.default_rng(semilla)
    for _ in range(cantidad):
        bloques = np.stack([generador.permutation(grupos[c])[:15] for c in clases])
        yield bloques[:, :5], bloques[:, 5:].reshape(-1)
```

```
def distancias(vectores, soporte, consultas, k):
    prototipos = F.normalize(vectores[soporte[:, :k]].mean(dim=1), dim=1)
    return torch.cdist(vectores[consultas], prototipos).numpy()


def evaluar(vectores, grupos, particion, k):
    cal = list(episodios(particion["calibracion"], grupos, 900, 10))
    test = list(episodios(particion["prueba"], grupos, 1000, 20))
    d_cal = np.concatenate([distancias(vectores, *e, k) for e in cal])
    verdad_cal = np.tile(np.repeat(np.arange(len(particion["calibracion"])), 10), len(cal
    ))
    coincide_cal = verdad_cal[:, None] == np.arange(d_cal.shape[1])
    far_cal, tar_cal, umbrales = roc_curve(coincide_cal.ravel(), -d_cal.ravel())
    umbral = float(-umbrales[np.argmax(tar_cal - far_cal)])

    d = np.concatenate([distancias(vectores, *e, k) for e in test])
    verdad = np.tile(np.repeat(np.arange(len(particion["prueba"])), 10), len(test))
    coincide = verdad[:, None] == np.arange(d.shape[1])
    aceptado = d <= umbral
    aciertos = int((d.argmin(axis=1) == verdad).sum())
    fa = int(aceptado[~coincide].sum())
    fr = int((~aceptado[coincide]).sum())
    return {
        "referencias": k, "umbral": umbral, "aciertos": aciertos,
        "consultas": len(verdad), "exactitud": aciertos / len(verdad),
        "fa": fa, "fr": fr, "negativas": int((~coincide).sum()),
        "positivas": int(coincide.sum()), "far": float(aceptado[~coincide].mean()),
        "frr": float((~aceptado[coincide]).mean()),
        "auc": float(roc_auc_score(coincide.ravel(), -d.ravel())),
    }


# %% Ejecucion y evidencia
def ejecutar(salida, cache=None):
    datos = fetch_lfw_people(
        data_home=cache, funneled=True, color=False, min_faces_per_person=20, resize=0.5
    )
    imagenes, grupos = preparar(datos)
    particion = partir(grupos)
    inventario = {
        fase: {"identidades": len(ids), "imagenes": sum(len(grupos[i]) for i in ids)}
        for fase, ids in particion.items()
    }
    print("LFW | identidad separada por fase", flush=True)
    for fase, fila in inventario.items():
        print(f"{fase:12} {fila['identidades']:3} identidades / {fila['imagenes']:4}
    imagenes")
    modelo = Codificador()
    historial = ajustar(modelo, imagenes, grupos, particion["ajuste"])
    modelo.eval()
    with torch.inference_mode():
        embeddings = torch.cat([modelo(bloque) for bloque in imagenes.split(128)])
        pixeles = F.normalize(imagenes.flatten(1), dim=1)
    filas = []
```

```
    for nombre, vectores in [("Pixeles", pixeles), ("Siamesa", embeddings)]:
        for k in (1, 5):
            fila = dict(modelo=nombre, **evaluar(vectores, grupos, particion, k))
            filas.append(fila)
            print(f"{nombre:8} k={k} | {fila['aciertos']:4}/{fila['consultas']} "
                  f"| exactitud {fila['exactitud']:.2%} | AUC {fila['auc']:.3f} "
                  f"| FAR {fila['far']:.2%} | FRR {fila['frr']:.2%}")

    salida.mkdir(parents=True, exist_ok=True)
    resumen = {
        "semilla": SEMILLA, "forma_origen": list(datos.images.shape),
        "particiones": inventario, "parametros": sum(p.numel() for p in modelo.parameters
    ()),
        "episodios_calibracion": 10, "episodios_prueba": 20,
        "historial": historial, "resultados": filas,
    }
    (salida / "resumen_lfw.json").write_text(
        json.dumps(resumen, ensure_ascii=False, indent=2, allow_nan=False), encoding="
    utf8"
    )
    fig, ejes = plt.subplots(1, 2, figsize=(9, 3.25), layout="constrained")
    ejes[0].plot(range(1, EPOCAS + 1), historial, color="#125e76", marker=".", lw=1.6)
    ejes[0].set(xlabel="Epoca", ylabel="Perdida media", title="Ajuste con Triplet Loss")
    for i, (nombre, color) in enumerate([("Pixeles", "#778899"), ("Siamesa", "#125e76")])
    :
        ys = [r["exactitud"] * 100 for r in filas if r["modelo"] == nombre]
        barras = ejes[1].bar(np.arange(2) + i * 0.32, ys, 0.30, color=color, label=nombre
    )
        ejes[1].bar_label(barras, fmt="%.1f", fontsize=8)
    ejes[1].set(xticks=[0.16, 1.16], xticklabels=["1 referencia", "5 referencias"],
                ylim=(0, 100), ylabel="Exactitud (%)", title="Identidades reservadas")
    ejes[1].legend(loc="upper left", frameon=False, fontsize=8)
    fig.savefig(salida / "resultados_lfw.png", dpi=180)
    plt.close(fig)
    print("Practica LFW finalizada.", flush=True)


if __name__ == "__main__":
    ejecutar(Path(__file__).resolve().parent / "resultados_lfw")
```